

Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



Verifiche di cybersicurezza per dispositivi
embedded

Tesi di Laurea

Laureando

Nenad Radulovic

Matricola 2101059

Relatore

Prof. Tullio Vardanega

ANNO ACCADEMICO 2025-2026

[da scrivere]

Ringraziamenti

[da scrivere]

Padova, settembre 2026

Nenad Radulovic

Sommario

Nell'ambito dello sviluppo di sistemi *embedded*, la verifica della corretta implementazione dei protocolli di comunicazione rappresenta un aspetto cruciale per garantirne l'affidabilità, in particolare nei settori *automotive* e industriale. La presente tesi descrive le attività svolte durante il mio *stage* universitario, incentrato sullo sviluppo di un *framework* di test per dispositivi *embedded* e sulla realizzazione di test per la verifica di specifici protocolli di comunicazione.

Durante il primo periodo, di diverse settimane, mi sono dedicato allo sviluppo, in linguaggio *Python*, di un *framework* per l'esecuzione automatizzata di test, che ha comportato sia il *refactoring* di test preesistenti sia la progettazione e la scrittura di nuovi casi di test. La maggior parte dei test realizzati era volta a verificare la comunicazione su canale *CAN* (*Controller Area Network*) e le sessioni diagnostiche *UDS* (*Unified Diagnostic Services*) su *CAN*, protocollo largamente diffuso nella diagnostica *automotive*.

Successivamente, ho esteso il progetto includendo la comunicazione *Modbus* su interfaccia *UART*, per la quale ho sviluppato test volti a verificare quali *function code* risultassero eseguibili dal dispositivo e su quali indirizzi. Per rendere possibile questa verifica è stato necessario, in uno stadio preliminare, programmare in linguaggio *C* una scheda *Nucleo-H533RE*, affinché fosse in grado di rispondere alle richieste *Modbus* e di implementare correttamente i *function code* previsti dal protocollo.

Nella tesi approfondisco inoltre le principali tecnologie e gli strumenti impiegati, discuto l'utilità dei test sviluppati e illustro le motivazioni alla base delle soluzioni progettuali adottate. Una sezione conclusiva è infine dedicata alle competenze acquisite durante l'esperienza, tra cui : competenze di programmazione in *Python* e in *C*, applicazione in ambito reale di concetti di progettazione del *software* e acquisizione di conoscenze sui protocolli *CAN*, *UDS* e *Modbus*.

Indice

1	Il contesto aziendale	1
1.1	<i>Bluewind</i> : aree di attività e mercati di riferimento	1
1.2	L'organizzazione aziendale e il metodo di lavoro	3
1.2.1	I ruoli	3
1.2.2	Il metodo <i>Agile</i>	5
1.3	I clienti	7
1.4	Lo <i>stack</i> tecnologico aziendale	8
1.4.1	Il linguaggio <i>C</i>	9
1.4.2	Il linguaggio <i>Assembly</i>	10
1.4.3	<i>PC-lint</i>	10
1.4.4	Strumenti proprietari di aziende <i>partner</i> come <i>Infineon</i> e <i>ST</i>	10
1.4.5	<i>StrictDoc</i>	11
1.4.6	<i>Gitlab</i>	11
1.4.7	Servizi e strumenti proprietari	12
1.5	Propensione all'innovazione	12
2	Descrizione del tirocinio	14
2.1	Gli stage all'interno del piano aziendale	14
2.2	La spinta normativa	14
2.3	Il progetto e la sua finalità	14
2.4	Gli obiettivi del progetto	15
2.5	Le ambizioni personali	15

3	Lo sviluppo del progetto	16
3.1	La metodologia e le tecnologie	16
3.1.1	Il metodo di lavoro	16
3.1.2	Le tecnologie utilizzate	16
3.2	Analisi	16
3.3	Progettazione	17
3.3.1	Introduzione	17
3.3.2	Le classi ‘ <i>Target</i> ’	17
3.3.3	Le classi ‘ <i>Test</i> ’	17
3.3.4	Creazione ed esecuzione	17
3.3.5	Persistenza	17
3.3.6	<i>API</i>	18
3.3.7	<i>Frontend</i>	18
3.4	Programmazione	18
3.5	Verifica e validazione	18
3.6	Resoconto del lavoro svolto	18
3.6.1	I risultati conseguiti	18
3.6.2	Migliorie possibili	19
4	Valutazione retrospettiva	20
4.1	Valutazione del raggiungimento degli obiettivi	20
4.2	La maturazione professionale	20
4.3	Confronto con il piano di studi	20
	Acronimi e abbreviazioni	i
	Glossario	ii

Elenco delle figure

1.1	Rappresentazione grafica di un esempio di gerarchia dei ruoli all'interno dell'azienda.	5
1.2	Schema che rappresenta la differenza di interazioni tra i clienti in ambito <i>automotive</i> e quelli in ambito industriale generale	8
1.3	Schema che rappresenta l'interazione delle tecnologie all'interno dei processi aziendali.	9

Elenco delle tabelle

Elenco dei codici sorgenti

Capitolo 1

Il contesto aziendale

Nel seguente capitolo presento il contesto organizzativo e produttivo dell'azienda Bluewind S.R.L., presso la quale ho svolto la mia attività di tirocinio. Durante la mia permanenza in azienda ho avuto modo di discutere con diversi colleghi sulla loro realtà lavorativa, e mi sono informato quanto più possibile sul tipo di attività che svolgono. Questo capitolo contiene le informazioni sull'organizzazione, i metodi di lavoro e la cultura aziendale di Bluewind che sono riuscito ad apprendere durante quest'esperienza.

1.1 **Bluewind:** aree di attività e mercati di riferimento

Fondata nel 1998, *Bluewind* è un'azienda informatica specializzata nel campo dei sistemi *embedded* e della sicurezza informatica. Si occupa sia dello sviluppo di prodotti *software* per aziende nell'ambito industriale, medico e dell'*automotive*, sia di fornire ad altre aziende servizi nell'ambito della sicurezza informatica e della *Functional Safety*_G.

Dal punto di vista dello sviluppo *software*, gli anni di esperienza nel settore hanno permesso all'azienda di accumulare un bagaglio di conoscenze e competenze che, ad oggi, gli permette di occuparsi dell'intero ciclo di vita di un

prodotto. L'azienda infatti non si occupa solo dell'implementazione del codice ma è coinvolta in tutti gli stadi del progetto:

- l'analisi dei requisiti
- la scelta e la progettazione dell'*hardware*
- la progettazione del *software*
- la stesura del codice
- lo sviluppo dei test
- le verifiche di sicurezza

Occorre tuttavia sottolineare che, in base alle richieste e necessità dei clienti, non necessariamente spetta sempre all'azienda occuparsi di ogni singolo passaggio di questo ciclo di vita.

Un'aspetto molto importante del settore in cui opera *Bluewind* è la presenza, per molti campi di utilizzo dei prodotti *embedded*, di normative e *standard* da rispettare per soddisfare i requisiti di sicurezza di alcuni settori critici. Per questo motivo *Bluewind* ha raccolto al suo interno una significativa conoscenza sulla *functional safety*, arrivando ad offrire servizi di consulenza e di validazione su questo tema ad aziende terze, e riuscendo a fare di questo ambito uno dei suoi principali punti di forza e competitività.

La disciplina della *functional safety* si occupa di garantire che un sistema si comporti in maniera prevedibile e che eventuali errori non portino al rischio di provocare danni.

Un caso pratico che si può prendere in esempio, per dimostrarne l'importanza, è quello di un impianto con vari macchinari ed componenti automatizzati e in movimento, all'interno del quale possono essere presenti degli operatori. In questo caso è fondamentale che il comportamento di tutte le componenti dell'impianto sia prevedibile, e che anche nell'eventualità di un errore *software*, o guasto *hardware*, non si arrivi a una situazione di rischio che possa provocare dei danni a persone o cose.

Questo è quindi un aspetto molto importante, e necessita che i requisiti delle

norme e degli *standard* tecnici vengano considerati all'interno del ciclo di vita di un prodotto fin dall'inizio.

Per accertare che durante la progettazione e lo sviluppo siano stati seguiti tali vincoli ci si affida a degli enti certificatori, che fanno una revisione della documentazione relativa al progetto e valutano che poi tutto sia coerente. Recentemente Bluewind ha accolto all'interno del suo *team* alcuni dipendenti che hanno già lavorato in passato come enti certificatori, in modo da poter rafforzare le proprie competenze nella materia.

Ad oggi, *Bluewind* è presente sul territorio con due sedi: una sede principale, a Castelfranco Veneto (VI), che si occupa delle attività di ricerca, sviluppo e *marketing*, e una sede secondaria a Milano focalizzata sulla *functional safety* e sulla sicurezza informatica in generale.

1.2 L'organizzazione aziendale e il metodo di lavoro

1.2.1 I ruoli

L'azienda è suddivisa in due macro reparti, ognuno dei quali è gestito da un responsabile : il reparto *Sales and Marketing*, incaricato degli aspetti commerciali, pubblicitari, di vendita dei prodotti, e il reparto *Research and Development* (*R&D*)_G che invece si occupa dell'analisi e sviluppo dei prodotti e della ricerca tecnologica.

Durante la mia attività di tirocinio sono stato inserito all'interno del reparto *R&D* e ho avuto modo di conoscere i principi alla base del loro metodo di lavoro.

All'interno del reparto, i dipendenti ricoprono diversi tipi di ruoli:

- **Analisti** : si occupano di studiare la realtà in cui si colloca il prodotto, analizzare il problema e stendere quali requisiti è necessario che il prodotto soddisfi, affinché possa adempiere al suo compito. Siccome la realizzazio-

ne di un progetto aziendale coinvolge, inevitabilmente, diverse materie, e ognuna di esse richiede specifiche conoscenze, gli analisti si dividono a loro volta in categorie in base allo specifico settore di competenza:

- **Analisti *hardware*** : siccome *Bluewind* fornisce e progetta componenti *hardware* su misura, ha nel suo *team* alcuni membri incaricati di capirne le necessità e valutare l'impatto che può avere il malfunzionamento di alcune componenti.
- **Analisti *software***
- **Analisti della sicurezza** : si differenziano per la loro conoscenza specifica nei temi della sicurezza informatica, e, nel particolare, delle norme e degli *standard* che ne regolamentano il tema
- **Sviluppatori *software*** : nel caso di *Bluewind* gli sviluppatori non svolgono solo le attività di stesura del codice, ma assumono anche il ruolo di progettisti, collaborando insieme agli analisti e ai clienti per interpretare al meglio i bisogni di quest'ultimi
- **Specialisti di *Functional Safety*** : data la rilevanza del tema della *functional safety*, all'interno dell'azienda vi è un gruppo di addetti che si occupa esclusivamente di questa materia
- ***Product Owner*** : i *Product Owner (PO)_G* sono i responsabili del progetto; si occupano di interfacciarsi con il cliente, gestire la parte di *report* delle attività, supervisionare il *team* di sviluppo, arrivando anche a prendere in carico aspetti burocratici e relativi alla fatturazione

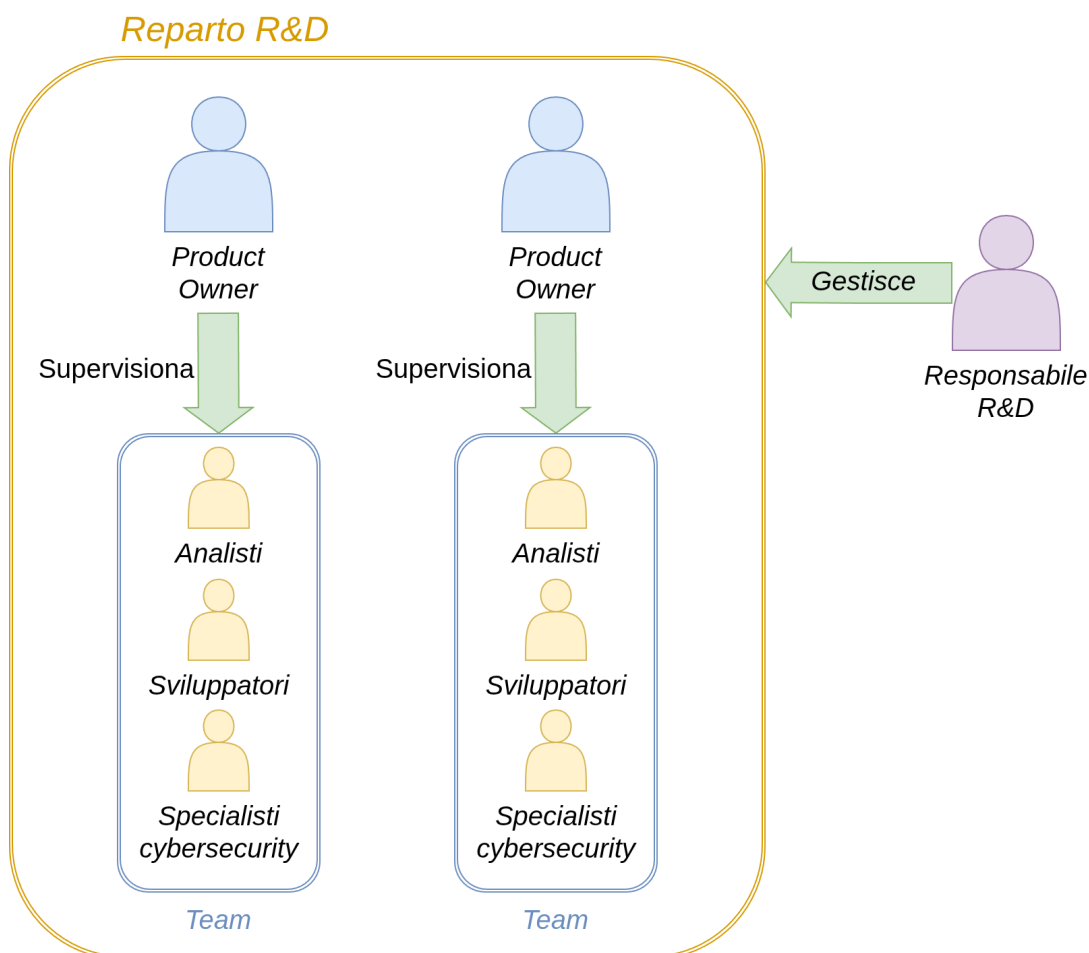


Figura 1.1: Rappresentazione grafica di un esempio di gerarchia dei ruoli all'interno dell'azienda.

Fonte: Elaborazione originale dell'autore

1.2.2 Il metodo *Agile*

All'interno di *Bluewind* ho avuto modo di osservare come l'adozione del metodo *Agile* non sia una prerogativa dello sviluppo *software*, inteso come il processo specifico di progettazione e implementazione di un prodotto, ma che i suoi principi possano integrarsi efficacemente anche all'interno di processi aziendali più generici e ampi.

Da quello che ho potuto cogliere, nell'organizzazione aziendale di *Bluewind* la metodologia *agile* non si limita a regolare il metodo di avanzamento dei progetti e delle attività dell' *R&D*, ma rappresenta anche un importante fondamento del loro modello economico.

Per l'organizzazione dei progetti, l'azienda adotta nello specifico il *framework Scrum_G*, dividendo i progetti in *sprint* di una o due settimane. I progetti sono assegnati a *team* di circa 3 persone (o più, in base al carico di lavoro), tra sviluppatori e analisti, con alla direzione un *product owner* che volge il ruolo di *scrum master*. Quest'ultimo si occupa quindi di agevolare la corretta applicazione del *framework scrum*.

In base alla necessità è possibile che alcuni dipendenti si occupino nello stesso periodo di più progetti in simultanea, tuttavia è una situazione che si cerca di evitare, spingendo su una suddivisione del lavoro più omogenea tra il personale.

All'inizio della collaborazione, il cliente acquista un determinato numero di *sprint* durante le quali il *team* di *Bluewind* si andrà a occuparsi del progetto. Agli sviluppatori e agli analisti viene dato l'incarico di definire le attività da svolgere per il proseguimento del progetto, valutando il tempo necessario per ognuna di esse.

In mancanza di una richiesta diversa, esplicita del cliente, una *sprint* si protrae per due settimane, con al suo interno diverse riunioni di sincronizzazione che coinvolgono sia il *team* di sviluppo di *Bluewind*, sia i rappresentanti della parte cliente. Non di rado accade che nelle riunioni di aggiornamento a confrontarsi sia il personale tecnico delle due parti. Questo permette al cliente di avere una visione molto più nel dettaglio di come procedono i lavori ed, eventualmente, di dare un *feedback* molto più diretto su come si preferisce che vengano gestite certe questioni. Coinvolgere il cliente ai vari stadi del progetto e motivarne la partecipazione, con una frequenza di riunioni che va oltre quella di un approccio *agile* classico, è uno dei principali impegni di *Bluewind*, che ha saputo dimostrare la sua efficacia.

Esaurito il numero di *sprint* a disposizione, il cliente può decidere se proseguire con lo sviluppo del progetto acquistandone altre, nel caso desiderasse aggiungere funzionalità o andare ad apportare migliorie. Nella stessa maniera viene gestito anche un eventuale piano di manutenzione e aggiornamento del prodotto.

A questo metodo di gestione del lavoro si legano poi anche altri processi azien-

dali, che non necessariamente sono coinvolti direttamente nell'evoluzione del prodotto, ma hanno la loro significativa rilevanza sia nella visione del progetto come sua intero, sia a livello di organizzazione aziendale.

1.3 I clienti

Occupandosi di sistemi *embedded*, i domini applicativi dei prodotti e dei servizi di *Bluewind* spaziano in molteplici mercati. Tra i suoi clienti una buona maggioranza arriva dai settori dell'*automotive*, dell'automazione industriale, dei sistemi di domotica e dell'elettronica di consumo.

Il modo con cui vengono mantenuti i rapporti e le interazioni con questi clienti può cambiare in base al loro settore di appartenenza. Un chiaro esempio è il mondo dell'*automotive*: all'interno dell'industria automobilistica esiste una gerarchia tra le aziende, che colloca ogni azienda in un livello specifico in base al suo ruolo nella produzione del prodotto. Al vertice della gerarchia si collocano le case automobilistiche, chiamate *Original Equipment Manufacturer (OEM)_G*, le quali decidono di delegare la produzione di alcune componenti ad altre aziende. L'azienda che riceve l'incarico dall'*OEM* prende il nome di azienda *tier 1*. Se successivamente essa decide di delegare, a sua volta, parte del lavoro ad altre aziende quest'ultime prenderanno il nome di *tier 2*. Lo stesso ragionamento si applica per ogni livello di delega che si aggiunge. Generalmente *Bluewind* si colloca tra il *tier 1* e il *tier 2*.

Nell'ambito industriale, invece, non avviene questo tipo di suddivisione, e l'azienda lavora direttamente con il produttore del sistema completo. Questo cambia il tipo di relazione che *Bluewind* ha con l'azienda committente e il progetto; nel caso di lavori svolti per delle *OEM*, *Bluewind* non ha modo di vedere come si integra il suo operato all'interno del progetto principale, in quanto la sua visione rimane limitata al singolo elemento che le è stato affidato. Nelle altre casistiche ha invece modo o di essere coinvolta direttamente con tutto il ciclo di vita del prodotto, qualora le venga commissionato l'intero processo, o quantomeno di

poter cogliere il funzionamento del sistema nel suo intero.

Tipologie di interazioni con i clienti

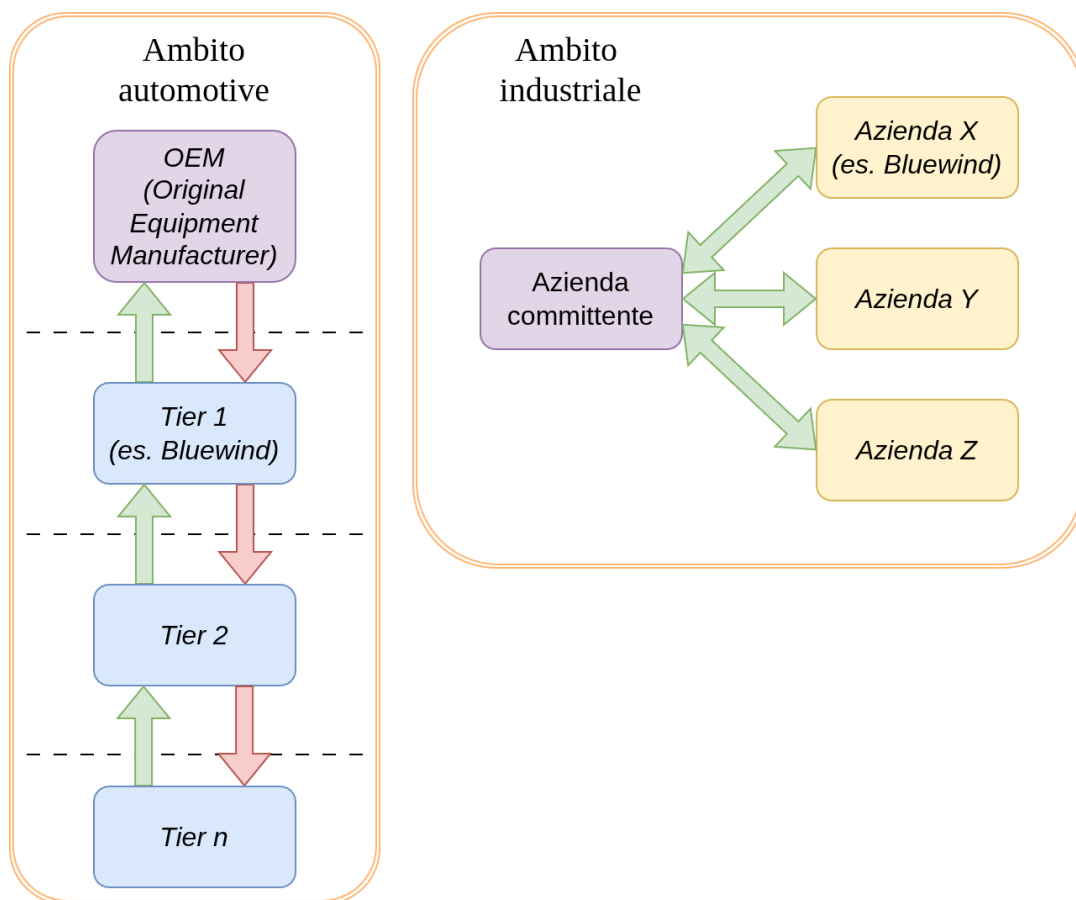


Figura 1.2: Schema che rappresenta la differenza di interazioni tra i clienti in ambito *automotive* e quelli in ambito industriale generale

Fonte: Elaborazione originale dell'autore

1.4 Lo *stack* tecnologico aziendale

Gli anni di attività hanno portato *Bluewind* a prendere confidenza con diverse tecnologie e strumenti, che spaziano in vari casi di utilizzo. La loro adozione tuttavia è fortemente influenzata dalle richieste del cliente. Difatti, *Bluewind* tende a non avere uno *standard* fisso su quali strumenti utilizzare, e si rifà molto a quello che viene utilizzato dall'azienda cliente. Questo avviene perché lavorando a stretto contatto con i propri clienti, e i rispettivi team di sviluppo, spesso su progetti già avviati, è necessario per *Bluewind* allinearsi con il metodo di lavoro del cliente. Spesso tale scelta è motivata anche dal fatto che il prodotto

finale comprende anche il codice sorgente, il quale rimane al cliente che potrebbe volerci lavorare ulteriormente in futuro.

Nonostante questa dinamica, vi sono delle tecnologie che a cui ricorrono con maggiore frequenza o alle quali, in mancanza di specifiche richieste, preferiscono affidarsi.

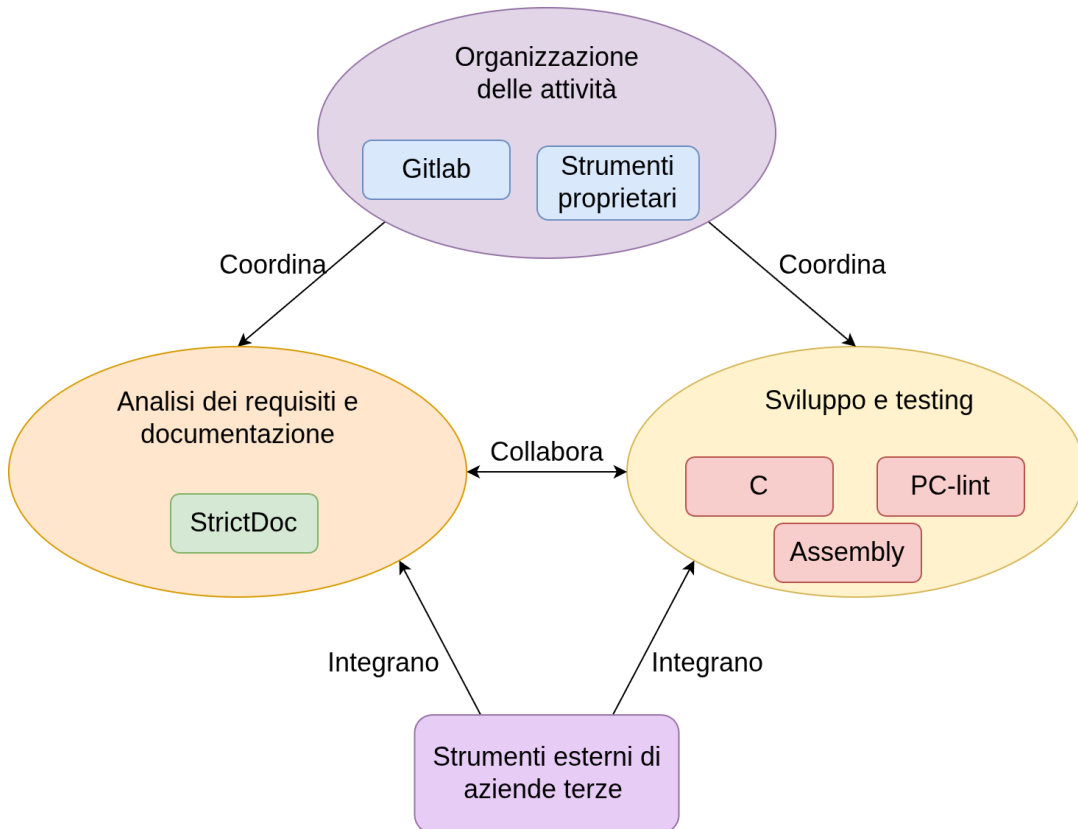


Figura 1.3: Schema che rappresenta l'interazione delle tecnologie all'interno dei processi aziendali.

Fonte: Elaborazione originale dell'autore

1.4.1 Il linguaggio *C*

Il linguaggio *C* rimane il linguaggio dominante nel settore *embedded*, in quanto offre un controllo quasi diretto sull'*hardware* (registri, memoria, interrupt) mantenendo comunque un livello di astrazione superiore all'*Assembly*. Su microcontrollori con risorse limitate (specialmente in termini di memoria, si parla anche pochi *kilobyte* di *RAM*) non c'è modo di adottare linguaggi più moderni con funzionalità più complesse come il *garbage collector* o l'*overhead*, occorre in-

vece qualcosa che offra prevedibilità nei tempi di esecuzione e controllo esplicito sull'allocazione. Su questo tema il linguaggio *C* rimane ancora oggi il punto di riferimento per il settore. Va detto però che questo controllo è anche notoriamente il suo punto debole: niente *memory safety* nativa, puntatori pericolosi, comportamenti *undefined* sono rischi che negli ambiti *safety-critical* (come quello *automotive*, o medico) vanno trattati con assoluto riguardo.

Nel capitolo successivo parlerò più a fondo delle regole *MISRA_C* che rappresentano un'importante *standard* per limitare gli errori più comuni.

1.4.2 Il linguaggio *Assembly*

Seppur non accada con grande frequenza, il *C* a volte non è sufficiente, e si rende necessaria l'adozione del linguaggio *Assembly* per raggiungere il livello di controllo richiesto. L'*Assembly* resta il linguaggio più basso disponibile nell'*embedded*, avendo una corrispondenza vicina all'uno a uno con le istruzioni macchina dell'architettura. Quindi viene utilizzato quando serve controllo assoluto sull'*hardware*, accesso a istruzioni specifiche non esposte dal *C* o un'ottimizzazione estrema di alcune funzioni critiche.

1.4.3 *PC-lint*

PC-lint è uno strumento di analisi statica per *C* e *C++* utilizzato in ambienti *safety-critical* per scoprire *bug*, vulnerabilità e verificare il rispetto delle regole imposte dagli *standard*, tra cui anche le regole *MISRA*.

È uno strumento *CLIC* e può essere integrato all'interno in un processo automatizzato di test e *deployment*.

1.4.4 Strumenti proprietari di aziende *partner* come *Infineon* e *ST*

Nell'ambito *embedded* è spesso richiesto l'utilizzo di componenti *hardware* costruite su misura per il loro caso di applicazione. Ne consegue che, di progetto in progetto, ogni dispositivo per cui andrà sviluppato il *software* sia leggermente

diverso, per le componenti (connettori, microprocessori, adattatori ecc.) di cui è composto. Per questo motivo, le aziende fornitrici di *hardware* come *Infineon Technologies* e *STMicroelectronics* (da notare che questi sono i *brand* che io ho avuto modo di conoscere nella mia, relativamente breve, esperienza, tuttavia lo stesso ragionamento si applica anche ad altri *partner* aziendali) mettono a disposizione delle schede che racchiudono un insieme predefinito di componenti che possono essere presenti nei dispositivi finali. In questo modo le aziende produttrici di *software* possono utilizzare le singole componenti di loro interesse, senza avere ancora a disposizione i dispositivi *hardware* definitivi. Questo semplifica lo sviluppo, rendendo più semplice per i programmatori ottenere un *feedback* sul funzionamento del codice.

Oltre alle schede, le aziende produttrici mettono a disposizione un gran catalogo di risorse per agevolarne l'utilizzo e la scrittura del codice, quali documentazione dettagliata sui prodotti, *IDE* specifici per il tipo di scheda e altri *tools* di monitoraggio e *debugging*.

1.4.5 *StrictDoc*

StrictoDoc è uno strumento gratuito e *open source* utilizzato per la scrittura di documentazione. Viene adottato specialmente per l'analisi dei requisiti in quanto permette di fare più facilmente riferimento tra i requisiti e al codice, di creare la matrice di tracciabilità e di esportare la documentazione in altri formati, anche quelli usati da *tools* proprietari. È uno strumento su cui *Bluewind* sta puntando molto per via della sua semplicità di utilizzo e della possibilità, in quanto *open source* di aggiungere funzionalità *custom* in base alla propria esigenza.

1.4.6 *Gitlab*

GitLab è una piattaforma che integra in un unico strumento la gestione del codice sorgente, il controllo di versione basato su *Git*, l'integrazione e distribuzione continua (*CI/CD*), e la gestione dei progetti. Si distingue inoltre da altre piattaforme concorrenti per la presenza di una versione *self-hosted*, rendendo-

la particolarmente apprezzata da aziende con esigenze specifiche di sicurezza e controllo sui propri dati.

Come ci si può aspettare, *Bluewind* adotta questo strumento integrandolo all'interno dei propri *server*, e rendendolo un elemento centrale all'interno della sua organizzazione aziendale. Questo strumento si integra molto bene con il metodo *agile* adottato dall'azienda, permettendo di organizzare, suddividere e tenere traccia delle attività in maniera più organizzata e meticolosa.

1.4.7 Servizi e strumenti proprietari

Oltre ad affidarsi a tecnologie e strumenti esterni, *Bluewind* ha una sua *suite* di strumenti proprietari per la gestione dei suoi processi interni. Lo scopo principale è quello di offrire servizi che agevolino alcuni compiti : l'organizzazione tra il personale, controllare la disponibilità dei vari dipendenti, organizzare incontri, gestire lo *smart working*, oppure, andando più sul tecnico, permettere di accedere da remoto a dispositivi *hardware* condivisi tra più utenti e utili allo sviluppo.

1.5 Propensione all'innovazione

In un periodo come quello attuale, in cui l'avanzamento tecnologico lascia poco spazio all'indecisione, avere una chiara idea e un chiaro metodo di approccio all'innovazione è fondamentale per potersi posizionare correttamente all'interno del mercato e per poter impiegare al meglio tempo e risorse. Per questo motivo all'interno di *Bluewind* è presente un impegno molto sentito verso il tema dell'innovazione che coinvolge tutto il personale.

Il loro specifico mercato di riferimento tuttavia non li aiuta: lavorare in settori dove la sicurezza e la prevedibilità dei sistemi sono aspetti profondamente critici, porta le aziende a dover avanzare con cautela nell'esplorare nuove opportunità. Un ostacolo concreto sono i vincoli posti dai requisiti obbligatori a

cui le aziende devono sottostare.

Un chiaro esempio di queste limitazioni è l'adozione dell'IA: l'utilizzo dei algoritmi di *machine learning*_G è ancora un argomento non del tutto esplorato vista la mancanza di regolamentazione a riguardo, ma soprattutto perché aggiunge una variabile imprevedibile all'interno di un contesto che spesso non se la può permettere.

Nonostante ciò ci sono stati dei passi avanti da parte dell'azienda anche su questo tema, e persiste il desiderio di approfondire anche questa tematica.

In generale, *Bluewind* segue un approccio molto attivo sul tema dell'innovazione, difatti ho notato in molti dipendenti la predisposizione ad accogliere le novità e a interessarsi fin da subito a come trarne vantaggio. Oltre a ciò l'azienda intraprende costantemente iniziative volte al miglioramento dei propri processi interni, sia tramite collaboratori esterni, sia con ricerche svolte dai dipendenti stessi.

Una parte importante in questo piano sono i tirocini studenteschi, i quali permettono a *Bluewind* di esplorare nuove tematiche e soluzioni per le quali altrimenti non ci sarebbero il tempo o le risorse da dedicare.

Capitolo 2

Descrizione del tirocinio

2.1 Gli stage all'interno del piano aziendale

In questa sezione parlo di come gli stage si integrino all'interno del piano aziendale e quali obiettivi l'azienda cerca di perseguire con essi.

2.2 La spinta normativa

Molte scelte e attività all'interno di *Bluewind* sono fortemente influenzate da fatto che nel loro settore vi sono diverse normative stringenti a cui le aziende devono sottostare. In questa sezione vorrei quindi spendere due parole per rendere nota la necessità dell'azienda di aderire a certi *standard*. Ritengo che sia un punto abbastanza importante quantomeno da introdurre per dare a chi legge una visione più chiara sul alcune dinamiche.

2.3 Il progetto e la sua finalità

In questa sezione parlo del progetto propostomi, da cosa nasce questa necessità e quale ruolo può avere all'interno del contesto aziendale.

2.4 Gli obiettivi del progetto

In questa sezione descrivo gli obiettivi prefissati con l'azienda da raggiungere durante l'attività di tirocinio.

2.5 Le ambizioni personali

In questa sezione parlo delle ragioni che mi hanno spinto a considerare questa proposta di stage, in relazione anche ai miei piani futuri di carriera. Oltre a ciò specificherò quali sono gli obiettivi personali che mi sono posto di raggiungere con quest'esperienza in maniera quanto più quantitativa possibile.

Capitolo 3

Lo sviluppo del progetto

3.1 La metodologia e le tecnologie

3.1.1 Il metodo di lavoro

In questo punto parlo del metodo di lavoro che ho adottato durante l'attività di stage per organizzare il lavoro e confrontarmi con il mio tutor.

3.1.2 Le tecnologie utilizzate

In questo punto descrivo quali tecnologie e strumenti (*software* e *hardware*) ho utilizzato per la progettazione e sviluppo del progetto.

3.2 Analisi

In questa sezione spiego il problema che sono andato ad affrontare e spiego la teoria che ci sta dietro. Nel particolare parlo dei principali protocolli utilizzati, spingendomi nel dettaglio quanto basta perché si possa comprendere più facilmente le sezioni successive. Oltre a ciò spiegherò la teoria che sta dietro i test implementati, motivando il loro scopo in relazione all'obiettivo del progetto.

3.3 Progettazione

3.3.1 Introduzione

Breve sezione di introduzione sull'architettura del progetto nel suo complesso.

3.3.2 Le classi ‘*Target*’

In questa sezione spiego come ho rappresentato i dispositivi *target* sui quali verranno lanciati i controlli di sicurezza.

3.3.3 Le classi ‘*Test*’

In questa sezione spiego come ho rappresentato i test da lanciare, partendo dai moduli che mi sono stati forniti e sui quali ho effettuato il *refactoring*. Questa parte riprende diversi concetti di §3.3, o per meglio dire ne rappresenta l'applicazione concreta all'interno del progetto.

3.3.4 Creazione ed esecuzione

In questa sezione spiego i servizi che si occupano della creazione e l'esecuzione delle classi di test. La creazione comprende una classe *factory* e un *pattern registry*; l'esecuzione comprende un *service* per l'esecuzione del singolo test e un altro per la gestione di più test eseguiti in serie. In merito a quest'ultimo parlerò anche della soluzione adottata per tenere traccia del *context* che permette ai test di prendere in *input* il risultato dei test eseguiti in precedenza.

3.3.5 Persistenza

In questa sezione spiego i servizi utilizzati per implementare uno storico dei test che conservi anche i risultati di quest'ultimi. Descriverò il *service* addetto alla persistenza e il *pattern repository* che interagisce con il *database SQLite*, motivando la scelta di quest'ultimo rispetto a un tradizionale *DBMS*.

3.3.6 *API*

In questa sezione spiego gli *endpoint* messi a disposizione dall'*API* implementata per accedere ai servizi del *framework*.

3.3.7 *Frontend*

In questa sezione spiego il ruolo del *frontend* come applicazione dimostrativa. È un punto un po' fuorviante, in quanto il senso del frontend è quello di dimostrare l'estendibilità del progetto ad altre componenti che non abbiano diretto accesso alla logica interna del *framework*.

3.4 Programmazione

In questa sezione parlo della stesura del codice ed eventuali difficoltà/incoerenze rispetto alla progettazione che ho incontrato. Parto parlando prima dello sviluppo in *Python* del *framework* e successivamente mi soffermo sullo sviluppo del *firmware* in *C*.

3.5 Verifica e validazione

In questa sezione parlo dei controlli effettuati per verificare la correttezza del codice. Mi soffermo sul motivare perché si non ho puntato a un alta copertura di codice e su quali punti mi sono concentrato.

3.6 Resoconto del lavoro svolto

3.6.1 I risultati conseguiti

In questa sezione discuto i risultati raggiunti sul piano qualitativo che su quello quantitativo, specificando i prodotti *software* e di documentazione che sono risultati alla fine del progetto. Faccio poi una presentazione del progetto completo

nel suo insieme, discutendone i diversi tipi di utilizzi che se ne possono fare dal punto di vista di un utente esterno.

3.6.2 Migliorie possibili

In questa sezione parlo di possibili migliorie che scoperto riguardando il progetto una volta finito. Cerco di dare una mia valutazione alle componenti del prodotto in relazione all'impegno che è servito per realizzarle, ne riconosco le limitazioni, spiego su cosa mi concentrerei se dovessi continuare a lavorarci. Discuto inoltre se vi sono scelte che farei diversamente se dovessi ricominciare da zero il progetto.

Capitolo 4

Valutazione retrospettiva

4.1 Valutazione del raggiungimento degli obiettivi

In questa sezione discuto i risultati ottenuti rispetto agli obiettivi prefissati, sia quelli del progetto che quelli personali.

4.2 La maturazione professionale

In questa sezione discuto l'esperienza lavorativa da me maturata e quali conoscenze ho acquisito in questo percorso.

4.3 Confronto con il piano di studi

In questa sezione discuto la differenza quali competenze del piano di studi mi hanno aiutato in quest'esperienza, quali mi mancavano e metto in evidenza alcune lacune de percorso di studi. Aggiungo delle mie opinioni sul piano di studi che ho sviluppato confrontandomi anche con alcuni colleghi in azienda e in relazione a quello che ho visto finora nelle mie esperienze lavorative.

Acronimi e abbreviazioni

CI/CD Continuous Integration/Continuous Delivery. [i](#), [11](#)

CLI Command Line Interface. [i](#), [10](#)

IDE Integrated Development Environment. [i](#), [11](#)

MISRA Motor Industry Software Reliability Association. [i](#), [10](#)

OEM Original Equipment Manufacturer. [i](#), [7](#)

PO Product Owner. [i](#), [4](#)

R&D Research and Development. [i](#), [3](#)

Glossario

Agile Approccio alla gestione di progetti e sviluppo *software* basato su cicli iterativi e incrementali, che privilegia flessibilità, collaborazione e adattamento al cambiamento rispetto alla pianificazione rigida. Fondato sui principi dell'Agile Manifesto (2001), promuove il coinvolgimento continuo del cliente, il rilascio frequente di valore funzionante e l'auto-organizzazione dei *team*. *Framework* come *Scrum* e *Kanban* ne rappresentano applicazioni pratiche diffuse. [i](#), [5](#)

CI/CD (Continuous Integration/Continuous Delivery) Pratica *DevOps* che automatizza le fasi di integrazione, test e rilascio del *software*. La *Continuous Integration* prevede l'unione frequente del codice in un *repository* condiviso con *build* e test automatici, per individuare errori precocemente. La *Continuous Delivery/Deployment* estende il processo automatizzando il rilascio in produzione, riducendo tempi e rischi legati alle *release* manuali. [i](#)

Embedded Indica un sistema integrato hardware-software progettato per svolgere una funzione specifica e dedicata all'interno di un dispositivo. I sistemi *embedded* operano tipicamente con risorse limitate (CPU, memoria, energia) e in ambiti come: centraline auto, elettrodomestici, dispositivi *IoT*, wearable e strumentazione medica. [i](#)

Functional Safety Ramo della sicurezza che si occupa di garantire il corretto comportamento di un sistema o uno strumento in seguito al cambiamento dei parametri di *input*, o al verificarsi di errori e/o guasti *hardware*. [i](#), [1](#)

IDE Ambiente di Sviluppo Integrato, *software* che riunisce in un'unica interfaccia gli strumenti necessari alla scrittura di codice. [i](#)

Machine Learning (ML) Branca dell'intelligenza artificiale che studia algoritmi capaci di apprendere pattern dai dati, migliorando le proprie prestazioni su un compito senza essere esplicitamente programmati per eseguirlo. [i](#), [13](#)

MISRA Organizzazione che pubblica linee guida di programmazione (principalmente per C e C++) volte a migliorare sicurezza, affidabilità e portabilità del codice, soprattutto in ambito *automotive* ed *embedded/safety-critical*. Gli *standard MISRA* definiscono regole e *best practice* per evitare costrutti ambigui o pericolosi del linguaggio, e sono spesso richiesti in contesti normativi o di certificazione. [i](#)

Open Source Prodotto *software* in cui il codice sorgente è reso pubblicamente disponibile, consentendo a chiunque di visualizzarlo, modificarlo e ridistribuirlo. [i](#), [11](#)

Product Owner Figura chiave nel metodo *Agile*, responsabile di massimizzare il valore del prodotto sviluppato dal *team*. Definisce la visione del prodotto, gestisce e priorizza le attività, e traduce le esigenze di *stakeholder* e clienti in requisiti chiari per il team. I suoi compiti possono variare in base all'organizzazione della realtà in cui si trova, andando a sovrapporsi in parte con quelli del *Project Manager*. [i](#)

Scrum *Framework* che applica i principi *Agile* per la gestione e lo sviluppo di prodotti complessi, organizzato in cicli temporali brevi e a durata fissa chiamati Sprint (tipicamente 2-4 settimane). Una figura chiave è lo *scrum master*, responsabile di facilitare l'adozione e la corretta applicazione del framework Scrum all'interno del team. Egli rimuove ostacoli che rallentano il lavoro, facilita le cerimonie *Scrum* (*Daily*, *Sprint Planning*, *Review*, *Retrospective*) e isola il *team* da interferenze esterne, promuovendo il miglioramento continuo dei processi. [i](#), [6](#)